

VisionEngine

Посматрајте кориснички интерфејс као корисник — рачунарски вид плус LLM вид за анализу и навигацију.

Сажетак

VisionEngine је одвојени Go алат који комбинује класични рачунарски вид са LLM заснованим видом како би анализирао корисничке интерфејсе, откривао UI елементе и визуелне проблеме, те градио навигационе графове прелаза између екрана апликације — са прикључивим вишекорисничким видовним позадинама и OpenCV-ом заштићеним иза ознаке за изградњу.

Кратак опис

Поново употребљив Go модул за анализу UI-а и изградњу навигационог графа. Нуди слој анализатора (UI елементи, разлике између екрана, визуелни проблеми), навигациони граф са BFS претрагом путева и извозом у DOT/JSON/Mermaid формату, као и LLM-визијске адаптере за GPT-4o, Claude, Gemini, Qwen-ВЛ и друге.

Детаљан опис

Већина аутоматизованих тестова корисничког интерфејса је у суштини слепа. Ослања се на стабла приступачности и DOM селекторе — машински појам интерфејса — и пропушта све оно што човек стварно доживљава: да ли је дугме видљиво рендеровано, да ли је распоред покварен, да ли је екран на који је стигао онај који је очекивао. VisionEngine премошћује ту празнину дајући аутоматизацији право опажање, способност да погледа UI и размишља о њему на начин на који би то учинила особа. Организован је у четири кооперативна слоја који се граде од сирових пиксела до разумевања целе апликације. **Анализатор** дефинише стабилан уговор — интерфејсе (Analyzer, VideoProcessor) и типове вредности (UIElement, ScreenAnalysis, ScreenDiff, Rect, Size, TextRegion,

VisualIssue, ScreenIdentity, Action, KeyFrame) са референтном имплементацијом StubAnalyzer — тако да корисници могу да откривају елементе, упоређују екране и откривају визуелне проблеме у односу на уговор који се неће мењати под њима. **Навигациони граф** подиже перспективу са појединачног екрана на целу апликацију, моделујући је као усмерени граф прелаза између екрана са BFS претрагом путева и три извозна формата (DOT, JSON, Mermaid), тако да аутоматизација не само да види екран, већ може да планира пут до било ког другог — а испоручује се са тестовима за стрес, аутоматизацију, интеграцију и безбедност како би се то доказало. Слој **LLM вида** додаје савремено мултимодално резоновање: интерфејс VisionProvider са адаптерима за OpenAI (GPT-4o), Anthropic (Claude), Gemini, Qwen-BL, Kimi, StepGUI, Астица и Ollama, састављен преко FallbackChain-a тако да неуспешан, ограничен бројем захтева или слабији провајдер отказује поступно, уместо да повуче цео процес са собом. Слој **Конфигурације** обрађује читавање и валидацију променљивих окружења, при чему се свака порука о грешци која је видљива кориснику прослеђује кроз `il8n.Translator`.

Одлука која све ово чини заиста примењивим јесте чињеница да тешка зависност од нативних библиотека није обавезна. OpenCV везе су заштићене ознаком за изградњу иза `-tags vision`, а подразумевана верзија испоручује се са стубовима — тако да цео модул може да се компајлира, тестира и покреће на било ком Go 1.25+ хосту без OpenCV алата у видокругу, а нативни стек се укључује само када корисник то експлицитно захтева. Управо то омогућава да се VisionEngine интегрише у обичан ЦИ покретач без потребе за прилагођеном сликом. Потпуно одвојен у складу са уставом (ЦОНСТ-051(Б)), укључује се у системе корисника — посебно у HelixQA — као подмодул једнаког кода, дајући тестирању UI-а заснованом на доказима прави пар очију.

Зашто смо га направили

Аутоматизација тестирања корисничког интерфејса која се ослања искључиво на стабла приступачности или селекторе пропушта оно што корисник заправо види. VisionEngine додаје праву визуелну интелигенцију — детекцију елемената, поређење екрана и закључивање засновано на LLM-визији — уз мапу навигације кроз екране апликације, тако да аутоматизација може и да опажа и да се креће кроз кориснички интерфејс.

Зашто је револуционарно

Спаја два иначе некомпатибилна приступа — брзу, детерминистичку класичну рачунарску визију и флексибилну, семантичку LLM-визију — иза јединственог интерфејса са ланцем резервних решења, тако да корисник добија прецизност једног и закључивање другог без потребе за избором. А задржавањем OpenCV-а као строго опционе компоненте, уклања се уобичајени трошак те моћи: сваки Go пројекат може стећи праву перцепцију корисничког интерфејса без увођења нативе визуелног алата у свој буилд процес.

Шта је иновативно

- Двострука перцепција: класична рачунарска визија (OpenCV/GoCV) плус LLM-визија са више провајдера и ланцем резервних решења.
- Граф навигације са BFS алгоритмом за проналажење путања и извозом у DOT/JSON/Mermaid форматима.
- OpenCV контролисан буилд-таговима како би модул остао изградив/тестабилан без нативе зависности.
- Потпуно одвојен, интернационализован, подмодул са истом кодном базом (користи га HelixQA).

Изазови и решења

- **Проблем са тешким нативе зависностима:** решено помоћу -tags vision гате-а и подразумеваних стубова како би ЦИ/хост системи без OpenCV-а и даље могли да се граде и тестирају.
- **Непоузданост провајдера визије:** решено интерфејсом VisionProvider и композитором FallbackChain.
- **Мапирање сложених токова апликације:** решено усмереним графом навигације уз BFS алгоритам за проналажење путања и извоз у више формата.
- **Спајање компонената:** решено помоћу ЦОНСТ-051(Б) децоуплинг-а и шава за интернационализацију.

Технолошки стацк (зашто + како)

- **Go (1.25+)** — језгро модула и сва четири слоја.
- **GoCV / OpenCV** — класична рачунарска визија, контролисана буилд-таговима.
- **Провајдери LLM-визије (GPT-4o, Claude, Gemini, Qwen-ВЛ, Kimi, StepGUI, Астица, Ollama)** — мултимодално закључивање о корисничком интерфејсу путем адаптера.

- **Графовски алгоритми (BFS)** — проналажење путања у навигацији.
- **ДОТ / JSON / Mermaid извозници** — визуализација графа навигације.
- **и18н преводилац** — одвојене корисничке стрингове.